

Sprint 14 Structured Notes: Normalization, SQL Injection, and N+1 Problem

1. Current Progress

You are now in **Sprint 14** of the Relational Databases learning path.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance

So far, you completed these stages:

Sprint 1: Tools and VS Code

You learned about:

- code editors and IDEs
- VS Code
- workspace
- project folders
- Live Server
- basic shortcuts and extensions

Simple meaning: Sprint 1 prepared your coding environment.

Sprint 2: Terminal, Shell, and Command Line Basics

You learned about:

- terminal
- command line
- shell
- Bash
- command history
- auto-complete
- clearing and interrupting commands

Simple meaning: Sprint 2 taught you what happens when you type commands.

Sprint 3: Bash File and Folder Commands

You learned how to:

- move between folders
- create files and folders
- rename, copy, and delete files
- read files
- use flags and options

Simple meaning: Sprint 3 taught you how to control files and folders from the terminal.

Sprint 4: Bash Scripting Basics

You learned about:

- Bash scripts
- `.sh` files
- shebang
- interpreter paths
- variables
- arrays
- loops
- `do` and `done`
- variable interpolation

Simple meaning: Sprint 4 taught you how to save terminal commands into reusable script files.

Sprint 5 to Sprint 10: Git Foundation

You learned:

- Git setup
- local Git workflow
- GitHub repositories and remotes
- branches and pull requests
- forks and sync
- `.gitignore`
- merge conflicts
- safer Git recovery tools
- beginner CI/CD ideas

Simple meaning: Sprints 5 to 10 taught you how developers track, share, review, and protect project changes.

Sprint 11: Relational Databases and PostgreSQL Setup

You learned:

- what a relational database is
- table, row, and column
- record, attribute, and field
- schema
- primary key and foreign key basics
- relational vs non-relational databases
- PostgreSQL
- pgAdmin
- psql
- PostgreSQL setup basics
- how to connect to the default `postgres` database

Simple meaning: Sprint 11 gave you the database foundation and PostgreSQL setup.

Sprint 12: SQL Basics

You learned:

- what SQL is
- SQL vs PostgreSQL
- statements and queries
- semicolons
- `psql` shortcut commands
- `CREATE DATABASE`
- `CREATE TABLE`
- snake_case naming
- data types
- `INSERT INTO`
- `VALUES`
- `SELECT`
- `WHERE`
- comparison operators such as `=` and `<`

Simple meaning: Sprint 12 taught you how to create simple tables, insert rows, and query basic data.

Sprint 13: Keys, Relationships, and Joins

You learned:

- primary key
- composite primary key
- foreign key
- orphaned records

- one-to-one
- one-to-many
- many-to-one
- many-to-many
- self-referencing relationships
- junction tables
- JOIN
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- SELF JOIN
- CROSS JOIN

Simple meaning: Sprint 13 taught you how to connect tables and read related data together.

Sprint 14: Normalization, SQL Injection, and N+1 Problem

Now you move into the final database-quality stage.

In Sprint 12, you learned how to build tables. In Sprint 13, you learned how to connect tables. In Sprint 14, you learn how to make database design **cleaner, safer, and faster**.

Simple meaning: Sprint 14 teaches you how to avoid messy tables, dangerous query building, and slow query patterns.

2. What Sprint 14 Is About

Sprint 14 is about three big ideas:

1. **Normalization**
2. **SQL injection prevention**
3. **N+1 problem avoidance**

Now you start learning:

- normalization
- redundancy
- data integrity
- insertion anomalies
- update anomalies
- deletion anomalies
- normal forms
- 1NF atomic values
- superkey
- candidate key

- minimality
- 2NF and partial dependency
- 3NF and transitive dependency
- BCNF / Normal Form 3.5
- SQL injection
- string concatenation risk
- parameterized queries
- input validation
- least privilege
- N+1 problem
- queries inside loops
- `JOIN` as a solution

Simple meaning: Sprint 14 teaches you how to design better tables, protect your queries, and avoid slow database access patterns.

3. Main Goal of Sprint 14

The main goal of Sprint 14 is:

Design safer and faster databases.

By the end of Sprint 14, you should be able to:

- explain what normalization means
- explain redundancy
- explain data integrity
- explain insertion, update, and deletion anomalies
- explain what 1NF, 2NF, 3NF, and BCNF mean
- explain what a superkey is
- explain what a candidate key is
- explain minimality
- explain partial dependency
- explain transitive dependency
- explain SQL injection
- explain why string concatenation is dangerous
- explain what parameterized queries do
- explain why input validation matters
- explain least privilege
- explain the N+1 problem
- explain why queries inside loops can be slow
- explain why `JOIN` can solve the N+1 problem

Simple meaning: You should be able to explain how to keep a database organized, secure, and efficient.

4. Why Sprint 14 Is Important

Sprint 14 is important because a database is not only about storing data. A good database should also be:

- organized
- accurate
- safe
- efficient

Without Sprint 14 ideas, a database can become:

- messy because data repeats too much
- unreliable because updates are inconsistent
- unsafe because attackers can change queries
- slow because the app sends too many small queries

Real projects need all three:

- clean table design
- safe query building
- efficient query behavior

Simple meaning: Sprint 14 is important because real databases must not only work. They must also work **correctly, safely, and fast**.

5. Big Beginner Idea

Think like this:

- **Normalization** = clean table design
- **SQL injection prevention** = safe query design
- **N+1 prevention** = efficient query design

Simple memory:

- normalization makes the database **clean**
- SQL injection prevention makes it **safe**
- N+1 prevention makes it **fast**

Simple meaning: Sprint 14 is about improving database quality from three directions.

Part 1: Normalization

6. What Is Normalization?

Normalization means organizing relational database tables using rules called **normal forms**.

The goal is to:

- reduce repeated data
- improve data integrity
- prevent data problems
- make tables more focused
- connect those focused tables using keys

A badly designed table mixes too many things together. A normalized design splits the data into cleaner, smaller tables.

Simple meaning: Normalization means splitting messy tables into better organized tables.

7. What Is Redundancy?

Redundancy means the same data is stored again and again in many places.

Example of redundancy

order_id	customer_name	customer_email	item
1	Amal	amal@email.com	Laptop
2	Amal	amal@email.com	Mouse
3	Amal	amal@email.com	Keyboard

In this table, the customer name and email repeat many times.

Why redundancy is bad

If Amal changes his email, you must update it in many rows. If you update some rows but forget one row, the database becomes inconsistent.

Simple meaning: Redundancy means unnecessary repetition of the same data.

8. What Is Data Integrity?

Data integrity means the data stays:

- correct
- consistent
- reliable

Bad example

customer_id	customer_name
1	Amal
1	Amal Silva

This is a problem because the same ID has conflicting data.

Better example

customer_id	customer_name
1	Amal Silva

Now the database is more trustworthy.

Simple meaning: Data integrity means the data is correct and does not contradict itself.

9. Why We Normalize

We normalize because poor design causes problems such as:

- repeated values
- confusing updates
- broken insert behavior
- accidental data loss
- unreliable results later

Normalization helps by:

- separating different kinds of information
- keeping each table focused
- reducing repetition
- improving integrity
- making relationships cleaner

Simple meaning: We normalize to make the database cleaner and more reliable.

10. What Are Anomalies?

An anomaly is a problem caused by poor table design.

Sprint 14 focuses on three main anomalies:

- insertion anomaly
- update anomaly
- deletion anomaly

Simple meaning: An anomaly is a data problem that happens because the table structure is bad.

11. Insertion Anomaly

An insertion anomaly happens when you cannot insert one kind of data without also needing some other data.

Example

Suppose a table stores customer details only inside order rows. If a new customer exists but has not placed an order yet, you cannot store that customer cleanly.

Simple meaning: Insertion anomaly means the design makes it hard to add new data.

12. Update Anomaly

An update anomaly happens when the same information must be updated in many places.

Example

If a customer email appears in 20 order rows, then changing the email means updating 20 rows. If one row is missed, the data becomes inconsistent.

Simple meaning: Update anomaly means one change must be repeated many times.

13. Deletion Anomaly

A deletion anomaly happens when deleting one row accidentally removes other important data.

Example

If a customer has only one order, and you delete that order row, you may also lose the only copy of the customer details.

Simple meaning: Deletion anomaly means removing one thing accidentally removes another important thing.

14. What Are Normal Forms?

Normal forms are rule sets used in normalization. Each higher normal form builds on earlier ones.

Main normal forms in Sprint 14:

- 1NF
- 2NF
- 3NF
- BCNF

Simple memory:

- 1NF = atomic values
- 2NF = remove partial dependency
- 3NF = remove transitive dependency
- BCNF = stricter version of 3NF

Simple meaning: Normal forms are the rules used to clean table design step by step.

15. First Normal Form (1NF)

1NF requires:

- atomic cells
- unique rows
- row order not being important
- column order not being important

What does atomic mean?

Atomic means one cell should hold one value, not multiple values together.

Bad example

customer_id	name	phone_numbers
1	Amal	0711111111, 0772222222

The `phone_numbers` cell contains multiple values. That breaks 1NF.

Better 1NF design

customers

customer_id	name
1	Amal

phone_numbers

customer_id	phone_number
1	0711111111
1	0772222222

Now each cell has one value.

Simple meaning: 1NF means one cell should contain one value only.

16. Superkey

A superkey is any column or set of columns that can uniquely identify a row.

Example

customer_id	name	email
1	Amal	amal@email.com
2	Nimal	nimal@email.com

Possible superkeys:

- `{customer_id}`
- `{email}`
- `{customer_id, name}`
- `{customer_id, name, email}`

All of these can identify a row uniquely. But some contain unnecessary extra columns.

Simple meaning: A superkey is any unique identifier, even if it includes extra columns.

17. Candidate Key

A candidate key is a **minimal** superkey. That means it is unique, but without unnecessary extra columns.

Example

If `customer_id` alone is enough, then:

- `{customer_id}` is a candidate key
- `{customer_id, name}` is not minimal

A table can have more than one candidate key. Then one of them is chosen as the primary key.

Simple meaning: A candidate key is the smallest useful unique identifier.

18. Minimality

Minimality means the key should contain only the columns truly needed to identify the row.

Example

If `email` alone is unique, then `{email, name}` is not minimal because `name` is extra.

This idea helps separate:

- superkeys
- candidate keys

Simple meaning: Minimality means do not add unnecessary columns to a key.

19. Second Normal Form (2NF)

2NF builds on 1NF. A table is in 2NF when:

- it is already in 1NF
- it has no partial dependency

Simple meaning: 2NF means non-key columns must depend on the full key, not only part of it.

20. What Is Partial Dependency?

A partial dependency happens when a non-key column depends on only part of a composite primary key.

This matters when the primary key uses more than one column.

Example

order_id	item_id	order_date	item_name	quantity
1	10	2026-06-02	Laptop	1
1	11	2026-06-02	Mouse	2

Assume the primary key is:

(order_id, item_id)

Problems:

- order_date depends only on order_id
- item_name depends only on item_id
- quantity depends on both order_id and item_id

That means the table has partial dependency.

Simple meaning: Partial dependency means a column depends on part of the combined key instead of the whole key.

21. 2NF Example Split

Bad combined table

order_id	item_id	order_date	item_name	quantity
1	10	2026-06-02	Laptop	1
1	11	2026-06-02	Mouse	2

Better 2NF design

orders

order_id	order_date
1	2026-06-02

items

item_id	item_name
10	Laptop
11	Mouse

order_items

order_id	item_id	quantity
1	10	1
1	11	2

Now each table has a clearer purpose.

Simple meaning: 2NF removes columns that depend on only part of a composite key.

22. Third Normal Form (3NF)

3NF builds on 2NF. A table is in 3NF when:

- it is already in 2NF
- it has no transitive dependency

Simple meaning: 3NF means non-key columns should not depend on other non-key columns.

23. What Is Transitive Dependency?

A transitive dependency happens when a non-key column depends on another non-key column, and that non-key column depends on the primary key.

Example

order_id	customer_id	customer_city	city_postal_code
1	100	Colombo	00100
2	101	Kandy	20000

Dependency chain idea:

`order_id → customer_id → customer_city → city_postal_code`

Problem: `city_postal_code` depends on `customer_city`, not directly on `order_id`.

Simple meaning: Transitive dependency means a non-key column depends indirectly on the key through another non-key column.

24. 3NF Example Split

Better 3NF design

orders

order_id	customer_id
1	100
2	101

customers

customer_id	customer_city
100	Colombo
101	Kandy

cities

city_name	city_postal_code
Colombo	00100
Kandy	20000

Now city data is stored separately.

Simple meaning: 3NF removes indirect dependencies between non-key columns.

25. BCNF / Normal Form 3.5

BCNF means **Boyce-Codd Normal Form**. It is sometimes called **Normal Form 3.5**.

BCNF requires:

- the table is already in 3NF
- every left-hand side of a functional dependency is a superkey

For beginners, the practical target is usually:

- understand 1NF well
- understand 2NF well
- understand 3NF well
- know that BCNF is stricter

Simple meaning: BCNF is a stronger, stricter version of 3NF.

26. Easy Memory for 1NF, 2NF, 3NF, BCNF

- **1NF** = no multiple values in one cell
- **2NF** = no partial dependency
- **3NF** = no transitive dependency
- **BCNF** = even stricter rule than 3NF

Simple meaning: Each new normal form removes another kind of design problem.

Part 2: SQL Injection

27. What Is SQL Injection?

SQL injection is a security vulnerability where an attacker changes or abuses a database query.

This can happen when user input is inserted into SQL unsafely.

Common places where it can happen:

- login forms
- search boxes
- URL parameters
- filter fields

Possible damage:

- bypass authentication
- steal data
- change data
- delete records

Simple meaning: SQL injection happens when user input becomes part of the SQL command in a dangerous way.

28. Why SQL Injection Happens

SQL injection often happens when developers build queries using:

- string concatenation
- string interpolation
- raw user input directly inside SQL

Unsafe idea

```
"SELECT * FROM users WHERE username = '" + username + "'" AND password = '" + password + "'"
```

This is dangerous because the database may treat user input as SQL syntax instead of normal data.

Simple meaning: SQL injection happens when a program mixes SQL code and user input carelessly.

29. String Concatenation Risk

String concatenation means joining text pieces together.

Example

If the application builds a query by joining strings, an attacker can type special SQL text into an input box.

That special text may change:

- the `WHERE` condition
- the meaning of the query
- the result that gets returned

Simple meaning: String concatenation is risky because it can allow user input to change the query itself.

30. Unsafe Login Example

Imagine a login system uses this SQL idea:

```
SELECT * FROM users
WHERE username = user_input
AND password = password_input;
```

If the app builds that query unsafely, an attacker may type:

```
' OR '1'='1' --
```

Why this is dangerous:

- `1=1` is always true
- `--` can comment out the rest
- the `WHERE` clause may become always true

That may let the attacker bypass the login.

Simple meaning: `OR 1=1` is dangerous because it can make the database treat the condition as always true.

31. What Does `--` Mean?

In SQL, `--` starts a comment in many SQL dialects. That means the rest of the line may be ignored.

Example idea

An attacker may try to make the real password check disappear by turning the rest of the query into a comment.

Simple meaning: `--` can hide the rest of the SQL line and help an injection attack succeed.

32. How to Prevent SQL Injection

Main prevention rule:

Use parameterized queries

Parameterized queries keep:

- SQL structure separate
- user data separate

That means the database treats the input as data, not as SQL commands.

Simple meaning: Parameterized queries stop user input from changing the query structure.

33. What Are Parameterized Queries?

A parameterized query uses placeholders for user values instead of directly placing the raw text into the SQL command.

The application sends:

- the SQL template
- the user values separately

Because of this separation, the database does not treat the input as executable SQL syntax.

Simple meaning: Parameterized queries tell the database, “This part is user data, not SQL code.”

34. Input Validation

Input validation means checking whether user input is acceptable before using it.

Examples:

- expected length
- allowed characters
- expected number format
- allowed sort values
- expected table or column choices from a safe list

Important beginner idea: Input validation is helpful, but it is a **secondary measure**. It should not replace parameterized queries.

Simple meaning: Input validation helps reduce bad input, but it is not the main protection by itself.

35. Table Names, Column Names, and Sort Order

A very important beginner rule: Do not directly take table names, column names, or sort directions from raw user input when possible.

Examples of risky input:

- table name from a search box
- column name from a URL parameter

- `ASC` / `DESC` order directly from raw input

If such choices are needed, they should come from a safe allow-list.

Simple meaning: Do not trust raw user input to choose SQL structure parts.

36. Least Privilege

Least privilege means giving the application only the permissions it truly needs.

Bad idea

A normal website login system uses a database account with full admin rights.

Better idea

The app uses a database account with only the rights needed for its real job.

Why this matters: If an attack happens, limited permissions reduce the damage.

Simple meaning: Least privilege means do not give the application more database power than necessary.

37. Easy SQL Injection Memory

Remember this pattern:

- raw user input + query string building = dangerous
- parameterized queries = safer
- input validation = helpful support
- least privilege = damage control

Simple meaning: Safe SQL means keeping SQL structure protected from user input.

Part 3: N+1 Problem

38. What Is the N+1 Problem?

The N+1 problem is a performance bottleneck caused by many small queries in sequence.

It usually looks like this:

- **1 query** gets a set of rows

- then **N more queries** get related data for each row

Example idea

- 1 query gets 50 orders
- 50 more queries get the customer for each order
- total = 51 queries

Simple meaning: The N+1 problem means one main query plus too many extra queries.

39. Why the N+1 Problem Happens

The N+1 problem often happens when a program uses:

- a loop
- a query inside the loop
- repeated lookup of related data one row at a time

Example pattern

```
get all orders
for each order:
    get customer
```

If there are many rows, the number of queries grows too much.

Simple meaning: N+1 often happens when the app asks the database for related data one row at a time inside a loop.

40. N+1 Example

First query:

```
SELECT * FROM orders LIMIT 50;
```

Then the program does this idea:

```
for each order:
    get customer for this order
```

That becomes:

```
1 query for orders
50 queries for customers
= 51 total queries
```

If the app loads 1,000 orders, then:

```
1 + 1000 = 1001 queries
```

That is inefficient.

Simple meaning: More rows can cause far too many database calls.

41. Why N+1 Is Bad

N+1 is bad because it can make an application:

- slower
- more wasteful
- harder to scale
- more expensive in database work

Even if each individual query is small, too many separate queries add overhead.

Simple meaning: N+1 is bad because many small queries together can become very slow.

42. JOIN as the Solution

A `JOIN` can often replace many small queries with one larger query.

Example

```
SELECT
  orders.order_id,
  orders.product,
  orders.quantity,
  customers.customer_id,
  customers.name AS customer_name,
  customers.email AS customer_email
FROM orders
```

```
JOIN customers
ON orders.customer_id = customers.customer_id;
```

Now one query gets the order data and related customer data together.

Simple meaning: A `JOIN` brings related data together in one query instead of many separate queries.

43. Queries in Loops

A very important beginner warning:

Queries inside loops often cause the N+1 problem.

That does not mean every loop is bad. It means you should be careful when the loop runs one SQL query for each item.

Bad pattern

```
for each product:
    query sales for that product
```

Better idea

Try to ask for the related data together using a `JOIN` or another grouped approach.

Simple meaning: Be careful when your code runs one query again and again inside a loop.

44. Easy N+1 Memory

Remember:

- `1` = first query to get a group of rows
- `N` = one extra query for each row

Safe idea:

- if related data is needed together, try to fetch it together
- `JOIN` is often the first solution to consider

Simple meaning: N+1 means too many repeated follow-up queries.

Part 4: How the Three Topics Connect

45. One Simple Summary Table

Topic	Main Problem	Main Solution
Normalization	messy repeated data	split tables properly
SQL injection	dangerous user input changes queries	use parameterized queries
N+1 problem	too many queries	use <code>JOIN</code> and avoid query-in-loop patterns

Simple meaning: Sprint 14 improves database quality in three ways: clean design, safe queries, and fast access.

46. Final Beginner Understanding

A strong beginner understanding of Sprint 14 is:

- normalization organizes tables
- SQL injection prevention protects query safety
- N+1 prevention improves performance

Together, these ideas help create databases that are:

- cleaner
- safer
- faster

Simple meaning: Sprint 14 teaches database quality, not only database basics.

Part 5: Practice Walkthrough

47. Practice Table

Use this messy table:

order_id	order_date	customer_id	customer_name	customer_city	city_postal_code	item_id	item_name
1	2026-06-02	100	Amal	Colombo	00100	10	Laptop

order_id	order_date	customer_id	customer_name	customer_city	city_postal_code	item_id	item_name
1	2026-06-02	100	Amal	Colombo	00100	11	Mouse
2	2026-06-03	101	Nimal	Kandy	20000	10	Laptop

48. Step 1: Identify Repeating Values

Repeated values include:

- customer_name
- customer_city
- city_postal_code
- item_name
- repeated order details for the same order

Simple meaning: The table mixes order data, customer data, city data, and item data in one place.

49. Step 2: Explain the Update Anomaly

If Amal changes city or email later, you may need to update multiple rows. If one row is missed, the database becomes inconsistent.

Simple meaning: Repeated data creates update anomaly risk.

50. Step 3: Convert Toward 1NF

If any cell contains multiple values, split them so each cell has one value. In this example, the main table already looks atomic by cell, but it is still too wide and too repetitive.

Simple meaning: 1NF is the first cleanup rule: one value per cell.

51. Step 4: Split Toward 2NF

Separate columns that depend only on part of a composite key.

Possible split:

orders

- order_id

- order_date
- customer_id

items

- item_id
- item_name

order_items

- order_id
- item_id
- quantity

Simple meaning: 2NF removes columns that depend on only part of a combined key.

52. Step 5: Split Toward 3NF

Separate indirect dependencies such as city and postal code.

Possible split:

customers

- customer_id
- customer_name
- customer_city

cities

- city_name
- city_postal_code

Now postal code is no longer stored repeatedly in customer or order rows.

Simple meaning: 3NF removes indirect dependency between non-key columns.

53. Step 6: Explain Why `OR 1=1` Is Dangerous

It is dangerous because:

- `1=1` is always true
- it can make the `WHERE` condition always true
- it can bypass intended filtering
- in login systems, it may bypass authentication checks

Simple meaning: `OR 1=1` can trick the database into returning rows it should not return.

54. Step 7: Explain How `JOIN` Helps With N+1

Instead of:

- 1 query for orders
- 1 extra query for each related customer

Use one joined query to get the order data and customer data together.

Simple meaning: `JOIN` reduces repeated follow-up queries.

Part 6: Common Beginner Mistakes

55. Mistake 1: Thinking Normalization Means Splitting Everything Too Much

Problem: A beginner may think every small detail must go into a new table immediately.

Better idea: Normalize carefully and logically. The common beginner target is usually up to **3NF**.

Simple meaning: Normalize enough to remove major problems, not randomly.

56. Mistake 2: Forgetting Why Redundancy Is Bad

Problem: Repeated values may look easy at first. Later they create update mistakes and inconsistency.

Simple meaning: Repeated data feels simple now but causes problems later.

57. Mistake 3: Confusing Candidate Key and Primary Key

Remember:

- candidate key = minimal unique identifier
- primary key = the chosen main candidate key

Simple meaning: A table may have multiple candidate keys, but one is chosen as the primary key.

58. Mistake 4: Thinking Input Validation Alone Stops SQL Injection

Problem: Validation helps, but it is not enough by itself.

Better habit: Use **parameterized queries first**, then input validation as support.

Simple meaning: Validation helps, but parameterized queries are the main defense.

59. Mistake 5: Giving the App Too Much Database Permission

Problem: If the app uses an admin-level account, an attack can cause more damage.

Better habit: Use **least privilege**.

Simple meaning: Limit the app's database permissions.

60. Mistake 6: Not Recognizing Queries in Loops

Problem: A beginner may write code that looks simple but actually creates N+1.

Better habit: Whenever you see a loop with a query inside it, stop and think.

Ask:

- can I get this data in one query?
- can I use a `JOIN`?

Simple meaning: Query-inside-loop patterns often deserve a second look.

Part 7: Sprint 14 Checkpoint

61. Sprint 14 Checkpoint

You pass Sprint 14 when you can explain:

- what normalization means
- what redundancy means
- what data integrity means
- what insertion anomaly means
- what update anomaly means
- what deletion anomaly means

- what 1NF means
- what 2NF means
- what 3NF means
- what BCNF means
- what a superkey is
- what a candidate key is
- what minimality means
- what partial dependency means
- what transitive dependency means
- what SQL injection is
- why string concatenation is dangerous
- what parameterized queries do
- why input validation matters
- what least privilege means
- what the N+1 problem is
- why queries in loops can be slow
- why `JOIN` helps solve N+1

Simple meaning: If you can explain these ideas clearly in your own words, you are ready to finish Sprint 14.

62. Good Beginner Checkpoint Answer

A good simple answer is:

Normalization helps organize data into cleaner tables and reduces repeated data. 1NF removes multi-value cells, 2NF removes partial dependency, and 3NF removes transitive dependency. BCNF is a stricter rule beyond 3NF. SQL injection is when user input changes the meaning of a query, so parameterized queries are safer than string building. Least privilege means an app should only get the database permissions it really needs. The N+1 problem happens when one query is followed by many small extra queries, often inside loops, and a JOIN can often fix that by bringing related data together in one query.

Part 8: Mini Assessment

63. Answer These Before Moving Forward

1. What is normalization?
2. What is redundancy?
3. What is data integrity?
4. What is an insertion anomaly?
5. What is an update anomaly?
6. What is a deletion anomaly?
7. What does 1NF require?
8. What is a superkey?

9. What is a candidate key?
 10. What does minimality mean?
 11. What is a partial dependency?
 12. What does 2NF remove?
 13. What is a transitive dependency?
 14. What does 3NF remove?
 15. Why is BCNF stricter than 3NF?
 16. What is SQL injection?
 17. Why is string concatenation risky?
 18. What do parameterized queries do?
 19. Why is input validation only a secondary measure?
 20. What does least privilege mean?
 21. What is the N+1 problem?
 22. Why do queries inside loops often cause N+1?
 23. How does a `JOIN` help solve the N+1 problem?
 24. Why is `OR 1=1` dangerous?
-

Part 9: Sprint 14 Completion Checklist

64. Tick These When Completed

- ☐ I can explain normalization in simple words.
 - ☐ I understand redundancy.
 - ☐ I understand data integrity.
 - ☐ I can explain insertion, update, and deletion anomalies.
 - ☐ I understand what 1NF requires.
 - ☐ I understand superkey, candidate key, and minimality.
 - ☐ I understand partial dependency.
 - ☐ I understand what 2NF removes.
 - ☐ I understand transitive dependency.
 - ☐ I understand what 3NF removes.
 - ☐ I understand that BCNF is stricter than 3NF.
 - ☐ I can explain SQL injection.
 - ☐ I understand why string concatenation is risky.
 - ☐ I understand what parameterized queries do.
 - ☐ I understand why input validation matters.
 - ☐ I understand least privilege.
 - ☐ I can explain the N+1 problem.
 - ☐ I understand why queries in loops are risky for performance.
 - ☐ I understand how `JOIN` can solve many N+1 cases.
 - ☐ I can explain why `OR 1=1` is dangerous.
 - ☐ I am ready to move forward after Sprint 14.
-

65. Final Sprint 14 Summary

Sprint 14 teaches three important database-quality ideas:

1. Normalization

This keeps tables clean and reduces repeated data.

2. SQL Injection Prevention

This keeps database queries safe from dangerous user input.

3. N+1 Problem Awareness

This keeps database access efficient by avoiding too many small queries.

Final simple meaning: A good database is not only able to store data. It should also store it in a clean way, access it in a safe way, and retrieve it in an efficient way.